

# Track 3: Programming Languages

June 2026

## Track 3: Programming Languages

MR ROKESH TECHNOLOGY — Training & Internship Programs

**Skill path:** Python / Java / C / C++ / Dart / JavaScript + OOP + DSA + Problem Solving

**Related:** [Master Syllabus](#) | [Duration Framework](#) | [Project Progression](#)

### Track Overview

**Who this is for:** Students preparing for coding interviews, competitive programming, backend roles, or language specialization.

**Prerequisites:** Basic computer literacy. 2-week crash requires picking **one primary language**.

**Language paths:** - **Python** — Data science crossover, rapid prototyping, interviews - **Java** — Enterprise, Android crossover, OOP depth - **C/C++** — Systems programming (6-month+ systems track) - **Dart** — Flutter crossover (see [Track 4](#)) - **JavaScript** — Full-stack crossover (see [Track 1](#))

**Tools:** VS Code, language SDK (Python 3.11+, JDK 17+, GCC), LeetCode/HackerRank accounts, Git/GitHub.

**Certification targets (9-month):** Oracle Java Foundations (optional); interview readiness primary goal

### Outcomes by Duration

| Duration | Hours   | Job Readiness                  | Key Deliverables                         |
|----------|---------|--------------------------------|------------------------------------------|
| 2 Weeks  | 40–50   | Syntax + basic DSA             | Calculator/Quiz app + 15 LC Easy         |
| 1 Month  | 80–100  | OOP + 50 LC Easy               | Expense tracker + 50 problems solved     |
| 3 Months | 250–300 | 150+ DSA problems              | Custom data structure library            |
| 6 Months | 500–600 | Dual language + 300+ DSA       | Splitwise clone / Pathfinding engine     |
| 9 Months | 750–900 | 500+ DSA + system design intro | Algorithm library + production app intro |

## Weekly Rhythm (All Durations)

Mon–Thu labs + DSA practice | Fri project/ problem set | Sat code review | Sun revision + GitHub

---

## 2-Week Crash Course (Pick 1 Primary Language)

**Mini Project:** Calculator / Quiz app | **Level:** L1

| Days  | Topics                                                 | Practice                     | Self-Check                         |
|-------|--------------------------------------------------------|------------------------------|------------------------------------|
| D1-2  | Syntax, variables, loops, functions, I/O               | 20 basic programs            | Write FizzBuzz from memory         |
| D3-4  | OOP: classes, inheritance, encapsulation, polymorphism | OOP mini library system      | Explain inheritance vs composition |
| D5-6  | Arrays, strings, file handling                         | File-based student records   | Big-O of linear search             |
| D7-8  | DSA basics: arrays, sorting, searching                 | 15 LeetCode Easy problems    | Implement binary search            |
| D9-10 | Recursion, problem-solving patterns                    | <b>Calculator / Quiz app</b> | Trace recursive Fibonacci          |

**Language-specific notes:** - **Python:** Use `dataclasses`, list comprehensions, `with` for file I/O - **Java:** Emphasize types, access modifiers, `ArrayList`, interfaces - **C++:** Pointers intro, STL vectors, basic memory concepts

---

## 1-Month Foundation

**Capstone:** Mini project + 50 LC Easy solved | **Level:** L2

| Week | Python/Java Track                                   | Project                               |
|------|-----------------------------------------------------|---------------------------------------|
| W1   | Syntax, OOP, collections (list/map/set)             | Student grade manager                 |
| W2   | File I/O, exception handling, modules/packages      | Expense tracker CLI                   |
| W3   | DSA: arrays, strings, recursion, sorting algorithms | Algorithm visualizer (basic)          |
| W4   | DSA: linked lists, stacks, queues + problem sprint  | <b>50 LC Easy + mini project demo</b> |

**W4 acceptance criteria:** - [ ] 50 LeetCode Easy problems logged in spreadsheet/GitHub - [ ] Expense tracker persists data to file - [ ] At least 2 sorting algorithms implemented from scratch

---

### 3-Month Job Ready

**Capstone:** Custom data structure library | **Level:** L3

| Month | Topics                                                              | Projects                             |
|-------|---------------------------------------------------------------------|--------------------------------------|
| M1    | Language mastery (Python OR Java), OOP deep dive, design principles | 2 CLI apps                           |
| M2    | DSA: arrays → trees → hashing (150+ problems)                       | Sorting/pathfinding visualizer       |
| M3    | Graphs, DP, greedy algorithms + interview prep                      | <b>Custom data structure library</b> |

**M3 Capstone acceptance criteria:** - [ ] Implement: Stack, Queue, LinkedList, BST, HashMap from scratch - [ ] Unit tests for each structure - [ ] Document time/space complexity for all operations - [ ] 150+ problems logged with pattern tags

---

## 6-Month Professional

**Dual language:** Python + Java (or C++ for systems track)

| Focus Area      | Content                                                      | Projects                                                    |
|-----------------|--------------------------------------------------------------|-------------------------------------------------------------|
| DSA roadmap     | 300+ problems: arrays → graphs → DP                          | Weekly problem sets                                         |
| OOP patterns    | Singleton, Factory, Observer                                 | Pattern demo projects                                       |
| Advanced topics | Multithreading, file systems, competitive programming basics | Splitwise clone / Inventory management / Pathfinding engine |

**Capstone acceptance criteria (L4):** - [ ] Production-quality CLI or desktop app with persistence - [ ] Clean OOP architecture with at least 2 design patterns - [ ] 300+ problems logged; mock interview score 70%+

---

## 9-Month Advanced

**Capstones:** Algorithm library + Production-grade application | **Level:** L5

**Additional modules:** C/C++ memory management + pointers, Dart for Flutter crossover, 500+ DSA (NeetCode 150 + Striver A2Z), system design basics, open-source contribution.

**Capstone 1 — Algorithm Library:** - [ ] 20+ algorithms with visualization - [ ] Benchmark suite comparing implementations - [ ] Published npm/PyPI package or GitHub release

**Capstone 2 — Production Application:** - [ ] Multi-module architecture with tests - [ ] CI/CD pipeline - [ ] System design doc for scaling

## Assessment Rubric (Track-Specific)

| Component      | Weight | Criteria                                  |
|----------------|--------|-------------------------------------------|
| Weekly Quiz    | 20%    | Syntax, OOP, algorithm concepts           |
| Lab Completion | 40%    | Programs run; DSA problems attempted      |
| Project        | 30%    | Code quality, architecture, test coverage |
| Portfolio      | 10%    | GitHub with problem log + project READMEs |

## Recommended Resources

- [LeetCode](#) / [NeetCode](#)
- [Striver A2Z DSA Sheet](#)
- Oracle Java Tutorials / Python Official Documentation

## Appendix: Mentor Notes

**Common pitfalls:** Memorizing solutions without understanding patterns; skipping edge cases; inconsistent naming conventions across languages.

**Stretch goals:** Contribute to open-source library; participate in CodeChef/LeetCode weekly contest.

**Cross-track:** [Track 1 — Full Stack](#); [Track 5 — AI/ML](#) for Python path.